

PER2-HALO: Data exploration and statistical testing

Tom Smith

2026-09-26

Contents

1 Purpose	1
2 Setup	2
3 Summarising to protein-level using <code>limpa</code>	3
4 Data exploration	6
4.1 Correlation between samples	6
4.2 Principle component analysis	6
5 Statistical testing	7
6 Visualisation of results	9
6.1 Volcano plot highlighting clock proteins and mTOR pathway proteins	9
6.2 Protein abundances for clock proteins and mTOR pathway proteins	10
7 Session info	13

1 Purpose

This notebook takes the filtered, normalised peptide-level data produced in notebook 1 and answers the biological question motivating the PER2-HALO experiment: does PER2 interact with mTORC1? It summarises peptide abundances to protein level using `limpa`’s Detection Probability Curve (DPC) model, tests for differential abundance between the PER2-HALO pulldown and the free HALO tag control using `limma`, and produces the manuscript volcano and summary plots highlighting circadian clock proteins and mTOR pathway components.

Because this is an immunoprecipitation (IP) experiment, the quantity of interest is *enrichment* in the PER2-HALO pulldown relative to the HALO-only control: proteins depleted in PER2-HALO relative to HALO are not biologically meaningful as PER2 interactors, so throughout the plots below “significant” is defined as 5% FDR *and* a positive fold change (higher abundance in PER2-HALO).

Input:

- `notebooks/output/HALO_1fq_qf.rds` from notebook 1 — the `QFeatures` object with the histone-normalised peptide-level data.
- `external/uniprot2details_unique.rds` from notebook 0 — UniProt protein annotations.
- `raw/HALO/PER2 IP - List for volcano plot.xlsx` — a manually curated list of mTOR pathway proteins to highlight, giving gene names and/or UniProt accessions.

Output:

- `results/HALO_wide_format_protein.tsv` and `results/HALO_long_format_protein.tsv` — the `limpa`-derived protein abundances, in wide and long format respectively.
- `results/HALO_limpa_results.tsv` — the differential abundance statistics (PER2-HALO vs HALO) for every protein.
- `results/plots/limpa_volcano_per2h_vs_halo.{png,pdf}` — the manuscript volcano plot.
- `results/plots/limpa_clock_mtor_per2h_vs_halo.{png,pdf}` — the manuscript summary plot of fold changes for clock and mTOR pathway proteins.

Analysis:

1. **Protein-level summarisation** — fit the DPC model and quantify protein abundances with `limpa::dpcQuant`, restricted to proteins with at least 2 peptides; fold the result back into the `QFeatures` object alongside its standard errors.
2. **Data exploration** — a sample correlation heatmap and PCA to confirm the two conditions separate as expected before proceeding to statistical testing.
3. **Statistical testing** — fit `~condition` with `limpa::dpcDE` and `limma::eBayes`, and extract the PER2-HALO vs HALO contrast for every protein.
4. **Visualisation** — a volcano plot highlighting circadian clock proteins and mTOR pathway proteins, a per-protein fold-change summary plot for the same highlighted proteins, and a per-sample abundance plot showing which values were quantified from real peptide measurements versus inferred by the DPC model in the absence of any peptide quantification.

How to read the results: positive `logFC` values indicate higher abundance in PER2-HALO relative to HALO (enrichment in the pulldown); significance is assessed at 5% FDR (`adj.P.Val < 0.05`). Where a protein has no peptide quantification in any HALO control replicate (e.g. CRY1/CRY2), its abundance and fold change are inferred entirely by the DPC model rather than measured, so these fold changes should be treated as directionally suggestive rather than precisely quantified; the per-sample abundance plot marks such values explicitly so they can be distinguished from directly quantified ones.

2 Setup

Read in the required packages

Define directory to save plots

```
plots_dir <- here('results/plots')
if(!dir.exists(plots_dir)){
  dir.create(plots_dir, recursive=TRUE)
}
```

Read in the quantification data.

```
lfq_qf <- readRDS(here('notebooks/output/HALO_lfq_qf.rds'))
uniprot2details_unique <- readRDS(here('external/uniprot2details_unique.rds'))
```

Define proteins to highlight in the figures below: `mtor_proteins_to_highlight` is read from a manually curated spreadsheet of mTOR pathway components (matched later by gene name or UniProt accession, since the spreadsheet provides both), and `clock_proteins` is a fixed list of core circadian clock genes.

```
mtor_proteins_to_highlight <- readxl::read_excel(
  here('raw/HALO/PER2 IP - List for volcano plot.xlsx'), col_names = FALSE)

clock_proteins <- c('PER2', 'PER1', 'CRY1', 'CRY2', 'CSNK1D',
  'CSNK1E', 'CLOCK', 'ARNTL', 'ARNTL1', 'ARNTL2')
```

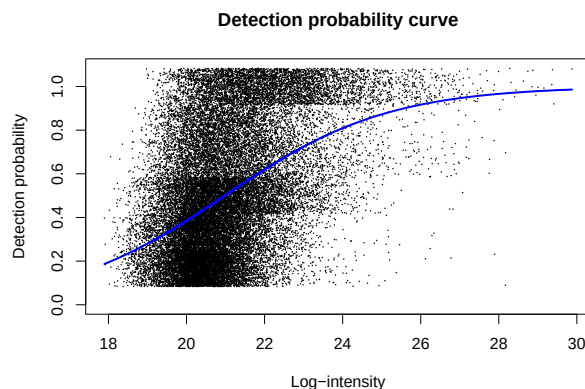
3 Summarising to protein-level using limpa

The first step is to perform protein quantification using `limpa`. We start by creating an `EList` object from the peptide level data, then fitting the Detection Probability Curve (DPC) model and performing protein quantification. The DPC is used to model the relationship between peptide detectability and abundance, allowing for more accurate protein quantification in the presence of missing values. We first restrict to proteins with at least 2 quantified peptides, since a protein's abundance and detectability cannot be reliably estimated from a single peptide.

```
y.peptide = new("EList", list(E=assay(lfq_qf[['peptides_norm_hist']]),
  genes=rowData(lfq_qf[['peptides_norm_hist']])))

# Filter to proteins with at least 2 peptides
y.peptide = filterCompoundProteins(y.peptide,
  protein.group = "Leading.razor.protein",
  min.n.peptides=2)

dpcfit <- dpc(y.peptide)
plotDPC(dpcfit)
```



```
y.protein <- dpcQuant(y.peptide, protein.id="Leading.razor.protein", dpc=dpcfit)
```

```
## Estimating hyperparameters ...
```

```
## Quantifying proteins ...
```

```
## Proteins: 1000 Peptides: 9032
```

```
## Proteins: 2000 Peptides: 19760
```

```
## Proteins: 3000 Peptides: 26526
```

```
## Proteins: 3959 Peptides: 33324
```

Next, we want to add the protein-level quantification results from `limpa` back into our `QFeatures` object for further analysis. For this, we need to create a 1) `summarizedExperiment` object, 2) Add it to our `QFeatures` object and 3) add assay links to the peptide-level data.

```
# Create a temporary protein-level summarizedExperiment so we can extract the
# rowData for our limpa summarizedExperiment
lfq_qf <- aggregateFeatures(lfq_qf,
                             i = "peptides_norm_hist",
                             fcol = "Leading.razor.protein",
                             name = "protein_sum",
                             fun = colSums, na.rm=TRUE)

lfq_qf <- QFeatures::zeroIsNA(lfq_qf, i='protein_sum')

# Create the limpa protein-level summarizedExperiment
limpa_protein <- SummarizedExperiment(
  assays=list(y.protein$E),
  rowData = rowData(lfq_qf[["protein_sum"]])[rownames(y.protein),],
  colData = colData(lfq_qf))

# Add the limpa protein-level assay to the QFeatures object
lfq_qf <- addAssay(x = lfq_qf,
                  y = limpa_protein,
                  name = "protein_limpa")

# Add assay link from peptide-level data to protein-level data
lfq_qf <- addAssayLink(object = lfq_qf,
                      from = "peptides_norm_hist",
                      to = "protein_limpa",
                      varFrom='Leading.razor.protein',
                      varTo='Leading.razor.protein')
```

Below, we extract the standard errors from the `limpa` results for later use in plotting, and build a long-format table of protein abundances flagging which values were directly quantified from peptides versus inferred by the DPC model without any peptide quantification in that sample (`is.imputed`).

```
# Extract standard errors and expression values in long format
stderrors_long <- y.protein$other$standard.error %>%
  data.frame() %>%
  tibble::rownames_to_column('UniprotID') %>%
  pivot_longer(cols=-'UniprotID', values_to='std_error', names_to='sample') %>%
  mutate(sample=remove_x(sample))
```

```

# Extract expression values in long format
exp_long <- y.protein$E %>%
  data.frame() %>%
  tibble::rownames_to_column('UniprotID') %>%
  pivot_longer(cols=-'UniprotID', values_to='exp', names_to='sample') %>%
  mutate(sample=remove_x(sample))

# Create long form data for plotting, including a column stating whether a protein
# quantification was 'imputed', which here means inferred without quantification
# of any peptides for the protein in that sample
long_protein <- biomasslmb::create_long_form_imputed_data(
  lfq_qf, i_imputed='protein_limpa', i_unimputed = 'protein_sum',
  row_variables = c('Leading.razor.protein', 'Gene.Names.First'),
  column_variables = c('condition', 'replicate'))

```

```
## Warning: 'experiments' dropped; see 'drops()'
```

```
## harmonizing input:
## removing 30 sampleMap rows not in names(experiments)
```

```

# Merge in the standard errors
long_protein_inc_std_error <- merge(
  long_protein,
  stderrors_long, by.x=c('primary', 'rowname'),
  by.y=c('sample', 'UniprotID'))

# Save long format protein quantification with standard errors
long_protein_inc_std_error %>%
  mutate(any_pep_quant=!is.imputed) %>%
  select(Leading.razor.protein, Gene.Names.First, condition, replicate, abundance=value, std_error, any)
  write.table(file = here('results/HALO_long_format_protein.tsv'),
    quote=FALSE, row.names=FALSE, sep='\t')

# We can now delete the temp SE
lfq_qf[['protein_sum']] <- NULL

```

```
## Warning: 'experiments' dropped; see 'drops()'
```

```
## harmonizing input:
## removing 6 sampleMap rows not in names(experiments)
```

Save the protein quantification in wide format

```

protein_wide_format <- assay(lfq_qf[['protein_limpa']])

protein_wide_format <- protein_wide_format %>%
  data.frame() %>%
  tibble::rownames_to_column('UniprotID')

write.table(protein_wide_format,
  file = here('results/HALO_wide_format_protein.tsv'),
  quote=FALSE, row.names=FALSE, sep='\t')

```

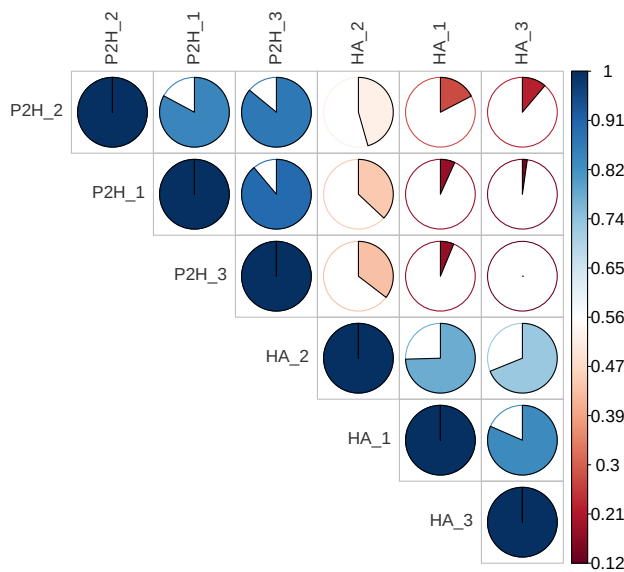
4 Data exploration

Before we proceed to statistical testing, we first explore the data to ensure that the experimental conditions have had the expected effect on the proteome.

4.1 Correlation between samples

We start with a plot of the correlation matrix, with the samples ordered by a hierarchical clustering of the distance between them. The samples clearly separate by condition

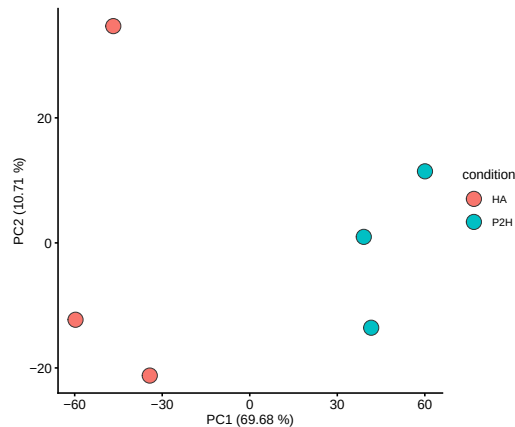
```
corrplot::corrplot(cor(assay(lfq_qf[['protein_limpa']]),use='complete',
                          method='spearman'),
  col = corrplot::COL2('RdBu'),
  method = 'pie',
  type = 'upper',
  order='hclust',
  tl.cex=1,
  cl.cex=1, tl.col='grey20', is.corr=FALSE)
```



4.2 Principle component analysis

Below, we use PCA to examine the major sources of variance.

```
plot_pca(lfq_qf, 'protein_limpa',
  scale=TRUE, center=TRUE) +
  aes(fill=condition) +
  #scale_shape_manual(name='', values=c(21,22)) +
  theme_biomasslmb(base_size=10)
```



PCA gives the same overall view of the samples, with the separation between conditions accounting for ~70% of the variance.

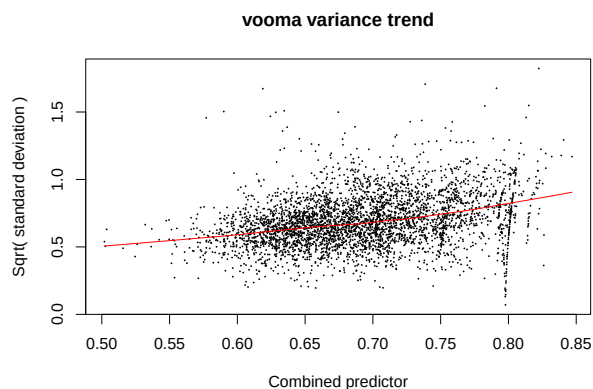
5 Statistical testing

Having established that the experimental conditions appear to have had the expected effect, we proceed to statistical testing for PER2-HALO (P2H) vs HALO (HA). We fit the model with `limpa::dpcDE`, which extends the DPC-based quantification to differential expression testing, and update it with `limma::eBayes` moderation, which shares information about the abundance-variance trend across all proteins to give more stable variance estimates for individual proteins (particularly useful here, where per-protein replicate numbers are small). `plotSA` is the standard QC plot for checking this mean-variance trend has been captured appropriately.

```
# Design matrix
condition <- factor(lfq_qf$condition, levels=c("HA", "P2H"))

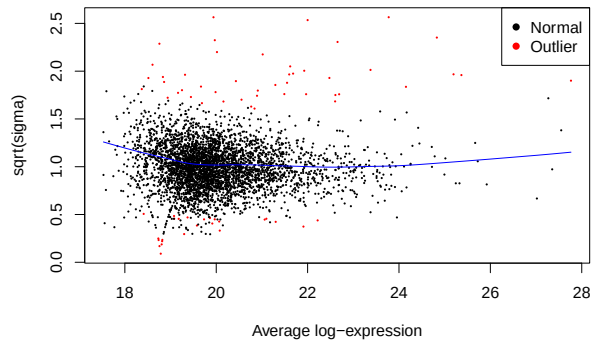
design <- model.matrix(~ condition)

fit <- dpcDE(y.protein, design)
```



```
## Update the model using the limma eBayes algorithm
final_model <- eBayes(fit = fit,
                      trend = TRUE,
                      robust = TRUE)
```

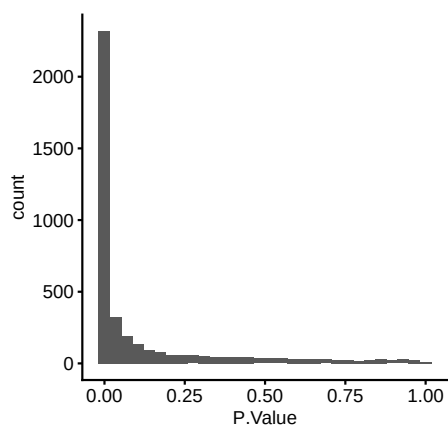
```
plotSA(final_model)
```



```
## Format results
limpa_results <- topTable(
  fit = final_model,
  coef = 2, confint = TRUE,
  adjust.method = "BH",    # Method for multiple hypothesis testing
  number = Inf)

# Check distribution of p-values
limpa_results %>%
  as_tibble() %>%
  ggplot(aes(x = P.Value)) +
  geom_histogram() +
  theme_biomasslmb()
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.
```



The p-value histogram is a standard diagnostic: under the null hypothesis of no difference between conditions, p-values should be uniformly distributed, with an excess near zero reflecting genuinely differentially abundant proteins.

How many significantly changing proteins at 5% FDR?

```
table(ifelse(limpa_results$logFC>0, 'Inc.', 'Dec.'),
      sig_5_perc_FDR=limpa_results$adj.P.Val<0.05)
```

```
##      sig_5_perc_FDR
##      FALSE TRUE
## Dec.    595  368
## Inc.    887 2109
```

6 Visualisation of results

6.1 Volcano plot highlighting clock proteins and mTOR pathway proteins

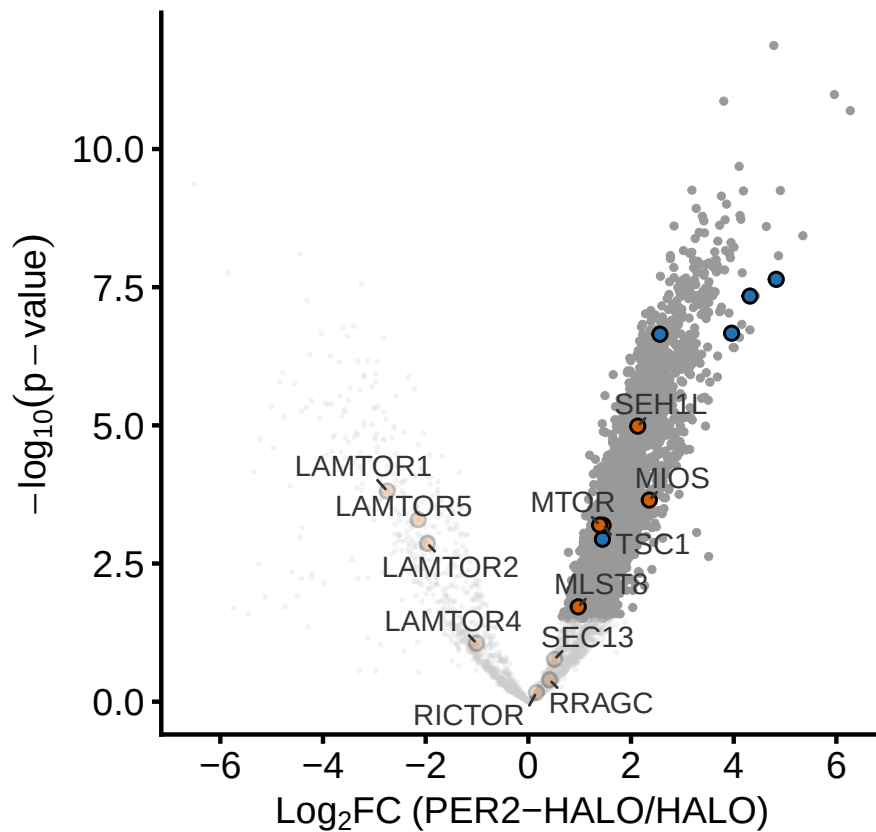
Below, we build the volcano plot that is the primary output of this notebook. Two sets of proteins are separately identified for highlighting and labelling: circadian clock proteins (`highlight_clock`, matched by gene name against the fixed `clock_proteins` list) and mTOR pathway proteins (`highlight_mtor`, matched by gene name or UniProt accession against the curated spreadsheet, since not every protein in that list has an unambiguous gene name match). As noted above, “significant” here requires both 5% FDR and enrichment in PER2-HALO ($\logFC > 0$), since depletion is not biologically informative for an interactor screen.

```
highlight_clock <- limpa_results %>%
  filter(Gene.Names.First %in% clock_proteins) %>%
  mutate(significant=adj.P.Val<0.05 & logFC>0)

highlight_mtor <- limpa_results %>%
  tibble::rownames_to_column('Leading.razor.protein') %>%
  filter(Gene.Names.First %in% union(mtor_proteins_to_highlight$`...1`,
                                     mtor_proteins_to_highlight$`...2`) |
        Leading.razor.protein %in% mtor_proteins_to_highlight$`...3`) %>%
  mutate(significant=adj.P.Val<0.05 & logFC>0)

p <- limpa_results %>%
  filter(is.finite(P.Value)) %>%
  mutate(significant=adj.P.Val<0.05 & logFC>0) %>%
  ggplot(aes(logFC, -log10(P.Value), alpha=significant, size=significant)) +
  geom_point(stroke=0.25, shape=20, aes(colour=significant)) +
  geom_point(size=2, pch=21, fill=get_cat_palette(1), data=highlight_clock) +
  geom_point(size=2, pch=21, fill=get_cat_palette(2)[2], data=highlight_mtor) +
  ggrepel::geom_text_repel(
    data=bind_rows(highlight_clock, highlight_mtor),
    aes(label=ifelse(Leading.razor.protein=='HALO_TAG', 'HALO_TAG',
                    gsub('.*', '', Gene.Names.y))),
    max.overlaps = 50, size=4, min.segment.length =unit(0, 'mm'),
    colour='grey20', alpha=1) +
  theme_biomasslmb() +
  scale_colour_manual(values=c('grey80', 'grey60'),guide=FALSE) +
  theme(strip.background = element_blank()) +
  scale_size_manual(values=c(1, 2), guide='none') +
  scale_alpha_manual(values=c(0.25, 1), guide='none') +
  labs(x=expression(Log[2]*FC~"(PER2-HALO/HALO)"),
       y=expression(-log[10] (p-value))) +
  scale_x_continuous(breaks=seq(-10,10,2))
```

```
print(p)
```



```
ggsave(here(plots_dir, 'limpa_volcano_per2h_vs_halo.png'))
ggsave(here(plots_dir, 'limpa_volcano_per2h_vs_halo.pdf'))

limpa_results %>%
  tibble::rownames_to_column('Leading.razor.protein') %>%
  select(Leading.razor.protein, Entry.Name, Protein.names, Gene.Names=Gene.Names.y,
         Gene.Names.First, NPeptides, logFC, CI.L, CI.R, AveExpr,
         P.Value, adj.P.Val) %>%
  write.table(here('results/HALO_limpa_results.tsv'), sep='\t',
             row.names=FALSE, quote=FALSE)
```

6.2 Protein abundances for clock proteins and mTOR pathway proteins

Having highlighted the proteins of interest on the volcano plot, we now summarise their fold changes in a dedicated plot, grouped into clock proteins, mTOR complex members (MTOR, MLST8, RICTOR), and the remaining mTOR pathway regulators. Proteins are ordered along the x-axis by their appearance in `to_plot` (itself ordered by significance, most PER2-HALO-enriched first), with PER2 always shown first.

```
to_plot <- limpa_results %>%
  tibble::rownames_to_column('Leading.razor.protein') %>%
```

```

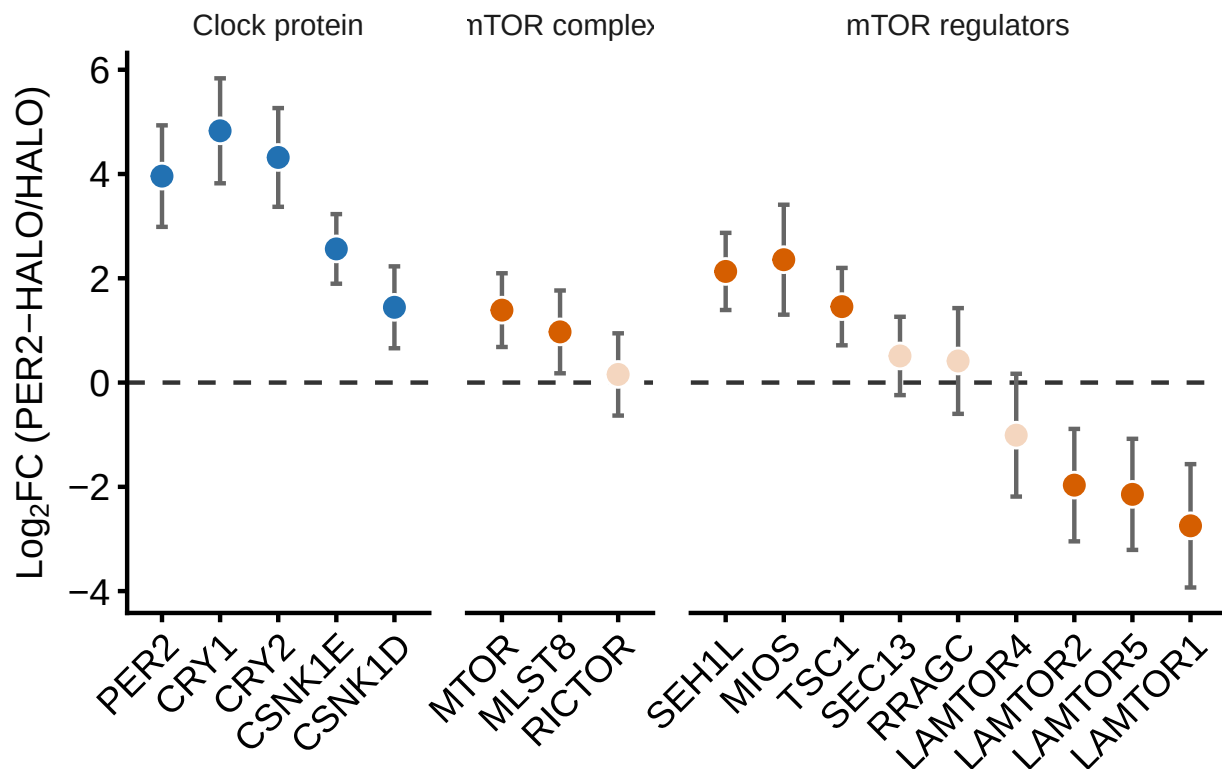
arrange(-sign(logFC)*-log10(P.Value)) %>%
mutate(rank=1:n()) %>%
filter((Gene.Names.First %in% clock_proteins)|
       Gene.Names.First %in% union(mtor_proteins_to_highlight$`...1`,
                                   mtor_proteins_to_highlight$`...2`)|
       (Leading.razor.protein %in% mtor_proteins_to_highlight$`...3`))

protein_order <- c('PER2', setdiff(sort(clock_proteins), 'PER2'))
protein_order <- c('PER2', setdiff(to_plot$Gene.Names.First, 'PER2'))

p <- to_plot %>%
mutate(Gene.Names.First=factor(Gene.Names.First,levels=protein_order)) %>%
mutate(type=case_when(
  Gene.Names.First %in% clock_proteins~'Clock protein',
  Gene.Names.First %in% c('MTOR', 'MLST8', 'RICTOR') ~ 'mTOR complex',
  .default = 'mTOR regulators')) %>%
mutate(type=factor(
  type, levels=c('Clock protein', 'mTOR complex', 'mTOR regulators')) %>%
ggplot(aes(Gene.Names.First, logFC)) +
geom_hline(yintercept = 0, linetype='dashed', colour='grey20') +
geom_errorbar(aes(ymin=CI.L, ymax=CI.R), width=0.2, colour='grey40') +
geom_point(colour='white', size=4) +
geom_point(aes(alpha=adj.P.Val<0.05, colour=type), size=3) +
theme_biomasslmb(aspect_square = FALSE) +
labs(x='',
      y=expression(Log[2]*FC~"(PER2-HALO/HALO)")) +
theme(axis.text.x = element_text(angle=45, vjust=1, hjust=1)) +

scale_colour_manual(values=c('grey80', get_cat_palette(5)[5]),
                    name='Significant\n(FDR<5%)') +
facet_grid(.~type, scales='free_x', space='free_x') +
theme(strip.background = element_blank()) +
scale_colour_manual(values=rep(get_cat_palette(2),
                               times=c(1,2)), guide='none') +
scale_alpha_manual(values=c(0.25, 1), guide='none') +
theme(panel.spacing.x = unit(5, 'mm'))
print(p)

```



```
ggsave(here(plots_dir, 'limpa_clock_mtor_per2h_vs_halo.png'))
ggsave(here(plots_dir, 'limpa_clock_mtor_per2h_vs_halo.pdf'))
```

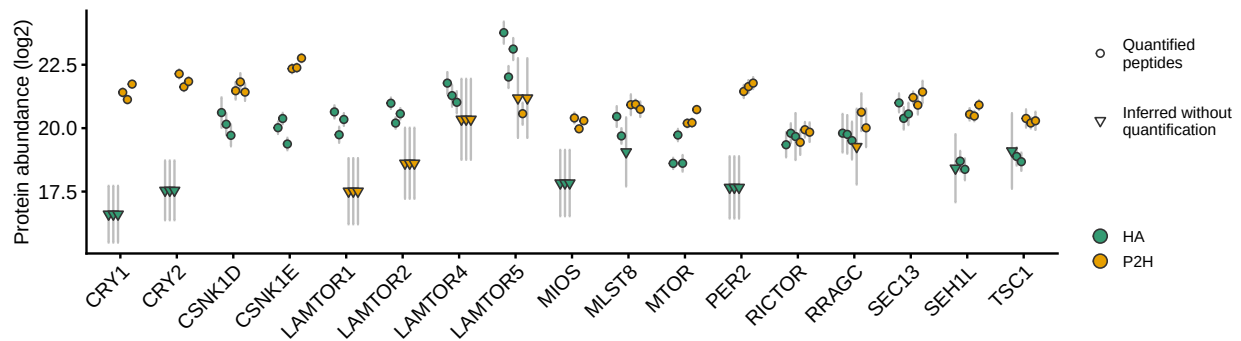
Below, we inspect the abundance values for individual samples for these proteins. For some like CRY1/CRY2, we haven't quantified any peptides in any HALO controls samples, so the protein abundance is inferred from the `limpa` model in the absence of quantification. The fold-changes in these cases are obviously less reliable, but given all the PER2-HALO samples are well quantified, it does suggest a strong enrichment. Aside from MIOS, all the mTOR pathway proteins highlighted are quantified in some peptides for at least 2/3 HALO control replicates, which gives us reasonable confidence in the fold-changes observed.

```
long_protein_inc_std_error %>%
  filter((Gene.Names.First %in% clock_proteins) |
         Gene.Names.First %in% union(mtor_proteins_to_highlight$`...1`,
                                     mtor_proteins_to_highlight$`...2`)) |
         (Leading.razor.protein %in% mtor_proteins_to_highlight$`...3`)) %>%
  mutate(Gene.Names.First=ifelse(
    Leading.razor.protein=='HALO_TAG', 'HALO_TAG-HALO', Gene.Names.First)) %>%
  ggplot(aes(Gene.Names.First, value, fill=condition,
             group=interaction(replicate, condition), shape=is.imputed)) +
  geom_errorbar(aes(ymin=value - std_error, ymax=value + std_error),
               position=position_dodge(width = 0.5), width=0.2, colour='grey') +
  geom_point(position=position_dodge(width = 0.5), size=2, colour='grey20') +
  theme_biomasslmb(border=FALSE) +
  theme(strip.background=element_blank(), plot.title=element_text(hjust=0.5)) +
  scale_shape_manual(values=c(21, 25),
```

```

      labels=c('\nQuantified\npeptides\n', '\nInferred without\nquantification\n'),
      name='') +
scale_fill_manual(values=get_cat_palette(4)[3:4], name='') +
labs(x='', y='Protein abundance (log2)' ) +
theme(axis.text.x = element_text(angle=45, vjust=1, hjust=1),
      aspect.ratio = 1/4) +
guides(fill=guide_legend(override.aes = list(size=4, pch=21)))

```



7 Session info

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_GB.UTF-8/en_GB.UTF-8/en_GB.UTF-8/C/en_GB.UTF-8/en_GB.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##

```

```

## other attached packages:
## [1] tidyr_1.3.2          dplyr_1.2.1
## [3] ggplot2_4.0.3        limpa_1.2.5
## [5] limma_3.66.0         here_1.0.2
## [7] biomasslmb_0.1.0     QFeatures_1.20.0
## [9] MultiAssayExperiment_1.36.2 SummarizedExperiment_1.40.0
## [11] Biobase_2.70.0       GenomicRanges_1.62.1
## [13] Seqinfo_1.0.0        IRanges_2.44.0
## [15] S4Vectors_0.48.1     BiocGenerics_0.56.0
## [17] generics_0.1.4       MatrixGenerics_1.22.0
## [19] matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
## [1] DBI_1.3.0             readxl_1.5.0.1       rlang_1.3.0
## [4] magrittr_2.0.5        clue_0.3-68          otel_0.2.0
## [7] compiler_4.5.2        RSQLite_3.53.3       systemfonts_1.3.2
## [10] png_0.1-9             vctrs_0.7.3          reshape2_1.4.5
## [13] stringr_1.6.0         ProtGenerics_1.42.0  pkgconfig_2.0.3
## [16] crayon_1.5.3          fastmap_1.2.0        backports_1.5.1
## [19] XVector_0.50.0        labeling_0.4.3       rmarkdown_2.32
## [22] visdat_0.6.0          ragg_1.5.2           purrr_1.2.2
## [25] bit_4.6.0             xfun_0.61            cachem_1.1.0
## [28] blob_1.3.0            DelayedArray_0.36.1  cluster_2.1.8.3
## [31] R6_2.6.1              stringi_1.8.9        RColorBrewer_1.1-3
## [34] genefilter_1.92.0     cellranger_1.1.0     Rcpp_1.1.2
## [37] knitr_1.52            BiocBaseUtils_1.12.0 Matrix_1.7-4
## [40] splines_4.5.2         igraph_2.3.3         tidyselect_1.2.1
## [43] abind_1.4-8           yaml_2.3.12          lattice_0.23-1
## [46] tibble_3.3.1          plyr_1.8.9           withr_3.0.3
## [49] KEGGREST_1.50.0       S7_0.2.2             evaluate_1.0.5
## [52] uniprotREST_1.0.0     survival_3.8-12      Biostrings_2.78.0
## [55] pillar_1.11.1         corrplot_0.95        checkmate_2.3.4
## [58] rprojroot_2.1.1       scales_1.4.0         xtable_1.8-8
## [61] glue_1.8.1            lazyeval_0.2.3       tools_4.5.2
## [64] robustbase_0.99-7     data.table_1.18.6.1  annotate_1.88.0
## [67] XML_3.99-0.24         grid_4.5.2           MsCoreUtils_1.22.1
## [70] AnnotationDbi_1.72.0  naniar_1.1.0         cli_3.6.6
## [73] textshaping_1.0.5     S4Arrays_1.10.1     AnnotationFilter_1.34.0
## [76] gtable_0.3.6          DEoptimR_1.2-1       digest_0.6.39
## [79] SparseArray_1.10.10   ggrepel_0.9.8        farver_2.1.2
## [82] memoise_2.0.1         htmltools_0.5.9      lifecycle_1.0.5
## [85] httr_1.4.9            statmod_1.5.2        bit64_4.8.6
## [88] MASS_7.3-65

```